Big Data(BCS061/BCDS-601/KOE-097

EduShine CLASSES
By- ARMAN ALI

Unit – 3 HDFS & Hadoop Environment

III	**HDFS (Hadoop Distributed File System):** Design of HDFS, HDFS concepts, benefits and challenges, file sizes, block sizes and block abstraction in HDFS, data replication, how does HDFS store, read, and write files, Java interfaces to HDFS, command line interface, Hadoop file system interfaces, data flow, data ingest with Flume and Scoop, Hadoop archives, Hadoop I/O: compression, serialization, Avro and file-based data structures. **Hadoop Environment:** Setting up a Hadoop cluster, cluster specification, cluster setup and installation, Hadoop configuration, security in Hadoop, administering Hadoop, HDFS monitoring & maintenance, Hadoop benchmarks, Hadoop in the cloud

Edushine Classes

Download Notes : https://rzp.io/rzp/MM5Q5BRS

EduShine CLASSES
By- ARMAN ALI

📁 Design of HDFS –

🔧 What is HDFS made for?

HDFS is designed to **store very large files** (like GBs or TBs).
It works across **many computers** connected in a network.
It is made for **fault tolerance**, **fast access**, and **low-cost hardware**.

❖ Main Parts of HDFS Design:

1. NameNode (Master Brain)

- Stores the **file names** and **locations of blocks**.
- Does **not store actual data**.
- Example: Like a librarian who knows where each book is kept.

2. DataNodes (Storage Workers)

- Stores the **actual data blocks**.
- Follows instructions from the NameNode.
- Example: Like shelves in a library holding books.

EduShine CLASSES By- ARMAN ALI

3. Blocks

- Every file is broken into **equal-sized parts** called blocks (128MB or 256MB).
- Each block is stored on different DataNodes.

4. Replication (Safety Backup)

Each block is **copied 3 times** (default) and saved on different machines.

If one machine fails, data is still safe on others.

5. Client (User Side)

- You (the user or program) interact with HDFS using a client.
- The client asks NameNode where the data is, then reads/writes directly to DataNodes.

⚙️ How HDFS Works (in steps):

✅ When you store (write) a file:

- File is split into blocks.
- NameNode decides where blocks should go.
- Blocks are stored on DataNodes (with 3 copies each).

✅ When you read a file:

- Client asks NameNode for block locations.
- Client reads blocks directly from DataNodes.

Big Data(BCS061/BCDS-601

EduShine CLASSES
By- ARMAN ALI

✅ Benefits of HDFS (Easy & Short)

- 📦 **Stores Huge Files** – Handles TBs and PBs of data easily
- 🔁 **Data Replication** – Makes 3 copies, so no data loss if 1 machine fails
- 💰 **Low-Cost Hardware** – Runs on normal (cheap) computers
- 📈 **Scalable** – You can add more machines anytime
- ⚙️ **Automatic Recovery** – Fixes problems like failed nodes on its own
- 🌐 **Distributed System** – Stores data across many machines

⚠️ Challenges of HDFS (Easy & Short)

- ❌ **Not good for small files** – Wastes memory and storage
- 🕒 **Not real-time** – Works better for batch (not instant) processing
- 💻 **Needs technical setup** – Not beginner-friendly to install/configure
- 🔌 **Depends on network** – Needs good connection between machines
- 🔍 **Requires monitoring** – Admins must keep track of health and performance

EduShine CLASSES
By- ARMAN ALI

File Size in HDFS – Easy Words

- HDFS is made to **store very large files** (like 1 GB, 100 GB, or even 1 TB).
- It works best when the file size is **big**, not small.
- **Small files** are not suitable because they waste memory and slow down the system.
- ⍰ **Example:** A 1 GB video file is good for HDFS. But 1000 small 1 MB files are not.

Block Size in HDFS – Easy Words

In HDFS, each file is split into **equal parts** called **blocks**.
The **default block size** is **128 MB** (sometimes 256 MB).
Each block is stored separately on different machines (DataNodes).
Blocks make it easy to **store and process big files**.

⍰ **Example:**

A 300 MB file will be split into 3 blocks:

- Block 1 → 128 MB
- Block 2 → 128 MB
- Block 3 → 44 MB

EduShine CLASSES
By- ARMAN ALI

🎁 What is Block Abstraction in HDFS?

In HDFS, **files are not stored as one big piece**.
Instead, each file is **broken into fixed-size parts** called **blocks** (like 128 MB each).
This breaking into blocks is called **block abstraction**.
HDFS **treats every file as a set of blocks** — not as a whole.

Example:

A 300 MB file → split into:
Block 1: 128 MB
Block 2: 128 MB
Block 3: 44 MB

✅ Benefits of Block Abstraction (Easy & Short)

1. 🔄 **Easier to store big files** – Can store parts across many machines
2. 💥 **Fault Tolerance** – If one block is lost, others are still safe
3. 📊 **Parallel Processing** – Blocks can be processed at the same time (faster results)
4. 💾 **Efficient storage** – Uses space on many computers
5. 🔁 **Replication made easy** – Each block can have 3 copies stored on different machines

EduShine CLASSES
By- ARMAN ALI

🔁 Data Replication in HDFS –

In HDFS, every block of a file is **copied multiple times**.
This is called **replication**.
By default, each block is stored **3 times** on **different machines** (DataNodes).
This helps in **data safety** and **fault tolerance**.

✅ Why Replication is Important?

🔒 **Data is safe** even if one machine fails
⚙️ **System keeps working** without stopping
🚀 **Faster access** – HDFS reads from the nearest copy

Example:

If a file has 2 blocks:
Block A → stored on 3 different computers
Block B → stored on another 3 different computers
So, total 6 blocks are saved.

EduShine CLASSES
By- ARMAN ALI

📥 How HDFS Stores Data :

- **File is split into blocks** (default 128 MB).
- **Blocks are stored** on different machines (DataNodes).
- **Each block is copied 3 times** (replication).
- **NameNode keeps record** of where blocks are stored.
- **Data is stored safely** and can be accessed anytime.

📝 How HDFS Write Operation Happens :

1.💻 **User gives a file** to HDFS (like data.txt)
2. HDFS **splits the file into blocks** (like 128 MB each)
3.🗺️ **NameNode decides** which DataNodes will store each block
4.🎁 **Blocks are saved** to DataNodes with **3 copies (replication)**
5.✅ After saving, **NameNode updates its record**
6.✅ File is now stored safely!

EduShine CLASSES
By- ARMAN ALI

📖 How HDFS Read Operation Happens :

1.💻 **User wants to read a file** (like data.txt)
2.🔍 User asks the **NameNode** – "Where are the blocks?"
3.🗺 NameNode tells the **locations of blocks**
4. Client directly **reads blocks from DataNodes**
5.If one copy fails, it reads from another
6.✅ File is read successfully!

☕ Java Interface to HDFS – Easy & Short

HDFS provides a **Java API** so that Java programs can **connect to HDFS**, **read**, **write**, and **manage files**.

Using this Java interface, you can write code to:

✅ Upload a file to HDFS
✅ Download a file from HDFS
✅ Delete or rename files
✅ Check file details

EduShine CLASSES
By- ARMAN ALI

📝 Short Note on Command Line Interface (CLI) in Hadoop –

The **Command Line Interface (CLI)** in Hadoop is a way to **talk to HDFS using simple text commands**.

You use it in a terminal (black screen) to:

- **Upload** files to HDFS
- **Download** files from HDFS
- **Check** what files are stored
- **Delete** or **move** files in HDFS

📝 Hadoop File System Interface –

The **Hadoop File System Interface** is like a **bridge** that lets Hadoop **work with different types of file systems**.

It helps Hadoop to:

- Read and write files not only from **HDFS**, but also from:
- **Local file system** (your computer)
- **Amazon S3** (cloud storage)
- **HBase** (a database)
- And more...

EduShine CLASSES
By- ARMAN ALI

What is Data Ingestion?

Data ingestion means **bringing data into a system** like Hadoop from different places.

☞ Example: Copying data from a database, website, or sensor into Hadoop so it can be stored and analyzed.

◆ Types of Data Ingestion:

1. **Batch ingestion** – Big chunks of data at one time (e.g., once a day)
2. **Real-time ingestion** – Data comes in continuously (e.g., live website clicks, sensors)

◆ Tools used:

1. **Flume** – for live data (like logs)
2. **Sqoop** – for moving data from databases (like MySQL)

◆ Challenges of Data Ingestion:

- Data comes too fast to handle
- Many different data formats
- Missing or wrong data
- Huge data size slows system

EduShine CLASSES
By- ARMAN ALI

What is Flume?

Apache Flume is a tool used to **bring live (real-time) data** into Hadoop, especially **log data** from websites, apps, or servers.

Example: If people are visiting a website, Flume can collect that activity (logs) and send it to Hadoop.

How Data Ingestion is Done in Flume :

Flume has **3 main parts**:

i. **Source** – Where the data comes from (like a web server or app logs)
ii. **Channel** – A middle area that holds the data for some time
iii. **Sink** – Where the data goes (like HDFS)

Example:

- A web server creates log files
- Flume collects logs using the **source**
- It holds the data in the **channel**
- Then sends it to Hadoop using the **sink**

EduShine CLASSES
By- ARMAN ALI

📝 What is Sqoop?

Sqoop (pronounced "Scoop") is a tool used to **move data between Hadoop and relational databases** like MySQL, Oracle, etc.

👉 **Example:** You have data in a MySQL database. Sqoop helps you **import** it into Hadoop or **export** it back to the database.

◆ How Data Ingestion is Done in Sqoop :

Sqoop works in **2 ways**:

i. **Import** – Bring data **from database to Hadoop** (HDFS, Hive, or HBase)

ii. **Export** – Send data **from Hadoop to database**

◆ How It Works:

- You run a **Sqoop command**
- It connects to the database
- It splits the data into parts (for fast transfer)
- It moves the data into HDFS or Hive tables

EduShine CLASSES
By- ARMAN ALI

📝 Short Note on Hadoop Archives :

Hadoop Archives (HAR) are used to **combine many small files into one big file** in Hadoop.

👉 Small files in Hadoop create problems (slow processing and waste of memory).

👉 HAR helps by grouping these files into a **single archive** to save space and improve performance.

◆ Limitation of Hadoop Archives :)

i. **Read-only** – You cannot change or add files in the archive.
ii. **Slower access** – Reading a file from HAR can be slower than reading a normal file.
iii. **No compression** – HAR does not compress files, so it doesn't save much space.
iv. **Extra processing** – Creating HAR takes time and CPU power.
v. **Not suitable for real-time data** – Works best for old files, not live data.

EduShine CLASSES
By- ARMAN ALI

📝 Compression in Hadoop :

Compression in Hadoop means **making files smaller** so they use **less space** and **move faster** in the system.

☞ Just like you zip a folder to save space on your computer, Hadoop compresses data to save space and speed up processing.

◆ Common Compression Formats in Hadoop:

Format	What It Does	Splittable?
Gzip	High compression	✗ No
Bzip2	Better compression, slower	✓ Yes
Snappy	Fast compression, less size	✗ No
LZO	Fast and splittable (needs setup)	✓ Yes

EduShine CLASSES
By- ARMAN ALI

📝 Short Note on Serialization in Hadoop :

Serialization means **changing data into a special format** so it can be saved or sent easily.
In Hadoop, serialization helps to:

- Convert data into bytes (small pieces)
- Send data over the network or save to files
- Make data easy to store and read later

📝 What is Avro?

Avro is a tool in Hadoop used for **storing and exchanging data** in a special format.
Think of Avro as a way to **pack data with its description** so anyone can read it easily, even if they use different systems.

◆ Features of Avro :

- **Stores data and its schema together**
  (Schema = description of data structure like fields and types)
- **Compact and fast** – makes data small and quick to read/write
- **Supports many programming languages** – Java, Python, C++, etc.

EduShine CLASSES
By- ARMAN ALI

◆ How Avro Works

- You define a **schema** (like a blueprint) for your data (example: name, age, city)
- Avro uses this schema to **convert data into a compact binary format**
- Data and schema are stored together or separately but linked
- When you read the data, Avro uses the schema to **understand and decode** it correctly
- If you change the schema later (like add a new field), Avro can still read old data without errors

📄 Cluster Specification –

A **cluster** in Hadoop means a group of **connected computers** (called nodes) that work together to store and process Big Data.

📋 Cluster Specification means:

👉 Deciding what kind of computers (hardware) and setup (software) we need in the cluster.

EduShine CLASSES
By- ARMAN ALI

✅ **It includes:**

1. **Number of Nodes**
   – How many computers will be in the cluster?
2. **RAM (Memory)**
   – How much memory each machine should have?
3. **Storage (Hard Disk)**
   – How much space we need to store data?
4. **CPU (Processor)**
   – How powerful should each machine be?
5. **Network**
   – How fast should the internet connection be between the computers?
6. **Master & Slave Role**
   – Which machine will be the **NameNode** (master)?
   – Which machines will be **DataNodes** (slaves)?

EduShine CLASSES
By- ARMAN ALI

⚙️ Cluster Setup and Installation –

Setting up a **Hadoop cluster** means preparing a group of computers to **work together** for storing and processing Big Data.

💻 Simple Steps to Setup a Hadoop Cluster:

✅ 1. Choose Computers (Nodes)

Decide which one will be the **master** (NameNode)
Others will be **workers/slaves** (DataNodes)

✅ 2. Install Java

Hadoop runs on Java, so you need to **install Java** on all machines.

✅ 3. Install Hadoop

Download and **install Hadoop** on all nodes (computers)

✅ 4. Set Up SSH (Passwordless Login)

Allows the master node to **connect to all workers** without password.

EduShine CLASSES
By- ARMAN ALI

✅ 5. Configure Hadoop Files

- Edit config files like:
  - core-site.xml
  - hdfs-site.xml
  - mapred-site.xml
  - yarn-site.xml

These files tell Hadoop how to run the cluster.

✅ 6. Format the NameNode

Run a command to **start the NameNode for the first time**

✅ 7. Start the Cluster

Use simple commands to **start Hadoop services**:

- start-dfs.sh
- start-yarn.sh

✅ 8. Check with Web Browser

Open Hadoop **web interface** to check if everything is working fine.

EduShine CLASSES
By- ARMAN ALI

⚙What is Hadoop Configuration?

Hadoop configuration means **setting up some files** to tell Hadoop **how to run**, **where to store data**, and **how machines will work together** in the cluster.

📂 Important Configuration Files

✅ 1. core-site.xml

- It tells the system the **main settings**.
- Example: NameNode address, file system type.

Like:

```
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://master:9000</value>
</property>
```

✅ 2. hdfs-site.xml

- It is used to configure **HDFS storage settings**.
- Example: where to store data, replication, block size.

Like:

```
<property>
  <name>dfs.replication</name>
  <value>3</value>
</property>
```

EduShine CLASSES
By- ARMAN ALI

✅ 3. mapred-site.xml

It configures **MapReduce settings**.

Example: which framework to use for job processing.

Like:

```
<property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
</property>
```

✅ 4. yarn-site.xml

It controls **YARN settings** (Resource Management).

Example: ResourceManager and NodeManager roles.

Like:

```
<property>
  <name>yarn.resourcemanager.hostname</name>
  <value>master</value>
</property>
```

EduShine CLASSES
By- ARMAN ALI

🔐 Security in Hadoop –

Hadoop security means **protecting data** stored in the cluster from **unauthorized access**.

✅ Main Security Features:

1. **Authentication** – Checks **who is trying to access** Hadoop (uses Kerberos).
2. **Authorization** – Decides **what the user is allowed to do** (read/write/delete).
3. **Encryption** – Keeps data **safe and private** while storing and moving.
4. **Audit Logs** – Keeps a **record of user actions** for checking later.

Hadoop security makes sure that **only trusted users** can use the system and **data stays safe**.

🔑 Delegation Token –

A **delegation token** is like a **temporary key** given to a user or application, so it can **access Hadoop services** (like HDFS) **without logging in again and again**.

EduShine CLASSES
By- ARMAN ALI

🔒 **It keeps things:**

- **Fast** (no repeated logins)
- **Safe** (token works only for a limited time)
- **Easy** (other apps or jobs can use this token)

📌 **Example:**

You log in once to Hadoop → get a **delegation token** → now your job (like data processing) can run for hours without asking for your password again.

📊 **HDFS Monitoring –**

HDFS Monitoring means **watching and checking** if the Hadoop file system (HDFS) is working **properly**.

✅ **Why is it needed?**

- To make sure **data is safe**
- To check if **nodes are active or failed**
- To track **storage usage** and **performance**

EduShine CLASSES
By- ARMAN ALI

🛠How to Monitor HDFS:

1.Web UI (Browser Dashboard)

- Access NameNode at: http://namenode-ip:9870/
- Shows live info: health, space, block reports, etc.

2.Command Line Tools

- Use commands like:
  - hdfs dfsadmin -report → shows all DataNodes
  - hdfs fsck / → checks file system health

3.Logs

- Check Hadoop logs for errors (/logs folder)

4.Third-party Tools

- Like **Ambari**, **Nagios**, **Ganglia** for advanced monitoring

EduShine CLASSES
By- ARMAN ALI

🛠Maintenance in Hadoop –

Maintenance in Hadoop means doing **regular checks and fixes** to keep the Hadoop system **healthy and working smoothly**.

✅ What Maintenance Includes:

1.Checking Disk Space

– Make sure there is enough storage on all nodes.

2.Monitoring Node Health

– Check if DataNodes or NameNode are working properly.

3.Replacing Failed Nodes

– If a machine (node) fails, replace or fix it.

4.Cleaning Logs

– Remove old logs to save space.

5.Balancing Data

– Use balancer tool to **evenly spread data** across nodes..

6.Upgrading Hadoop

– Install updates for better features and security.

7.Backup NameNode Data

– Keep a **copy of NameNode metadata** (FsImage & Edit Logs) for recovery

EduShine CLASSES
By- ARMAN ALI

📈 Hadoop Benchmarks –

Hadoop Benchmarking means **testing the performance** of a Hadoop system — like how fast it can process data or how well it stores files.

✅ Why Use Benchmarks?

- To check if the cluster is **fast and working properly**
- To **compare performance** before and after changes
- To find and fix **bottlenecks (slow parts)**

🛠Common Hadoop Benchmark Tools:

1. **TestDFSIO**
   – Tests HDFS speed: how fast it can **read/write files**
2. **TeraSort**
   – Tests **MapReduce** performance by sorting large data
3. **NNBench**
   – Tests **NameNode performance** (metadata speed)
4. **MRBench**: Tests **MapReduce job** performance on small data

EduShine CLASSES
By- ARMAN ALI

☁What is Hadoop in the Cloud?

Hadoop in the Cloud means running Hadoop on **cloud platforms** like AWS, Google Cloud, or Azure **instead of your own physical computers**.

So, you don't buy servers. You **rent cloud machines** to store and process big data using Hadoop.

✅ Reasons to Run Hadoop in the Cloud:

1. **No need to buy hardware**
   → Cloud gives ready-made servers.
2. **Scalable**
   → You can easily add or remove machines as needed.
3. **Cost-efficient**
   → Pay only for what you use.
4. **Faster setup**
   → No need to build your own data center.

EduShine CLASSES
By- ARMAN ALI

5. Easy maintenance

→ Cloud providers manage many things for you.

6. Global access

→ Access your Hadoop cluster from anywhere.

EduShine CLASSES
By- ARMAN ALI

Thank You….

Download Notes : https://rzp.io/rzp/MM5Q5BRS